

# Understanding and Detecting Scalability Faults in Large-Scale Distributed Systems

Hao-Nan Zhu\*, Goodness Ayinmode\*, Cesar A. Stuardo, Haryadi S. Gunawi, and Cindy Rubio-González

**Abstract**—Scalable distributed systems form the backbone of modern computing infrastructure. However, as scale grows, system complexity may lead to scalability faults. Scalability faults are challenging to uncover and diagnose, as they are often latent and only manifest at large-scale deployment. In this paper, we present the first comprehensive study on scalability faults and propose an approach for their detection. First, we systematically investigate 444 scalability issue reports from 10 large-scale distributed systems to understand the common anti-patterns and root causes of scalability faults. We found that the majority of these faults are caused by the synergy between dimensional code fragments and anti-patterns associated with them. Second, based on our findings, we design and implement SCALENS, a novel approach to detect scalability faults. SCALENS combines dynamic and static analyses to pinpoint dimensional code fragments and match them with anti-patterns. Our evaluation shows that SCALENS detects 4.2× more dimensional code fragments associated with known scalability faults compared to the baseline. On the latest stable versions of Cassandra, HDFS, and Ignite, SCALENS detects 334 dimensional code fragments with confirmed problematic behavior.

**Index Terms**—Scalability, distributed systems, fault detection, program analysis.

## I. INTRODUCTION

GIVEN the constraints of Moore’s Law and Dennard scaling, coupled with the escalating demand for computing power, the last decade has seen unprecedented deployment scales: over 300,000 Cassandra nodes are hosting over 100 petabytes of data at Apple, tens of thousands of MapReduce/Spark jobs are running on the Hadoop clusters of X (Twitter), and trillions of Kafka messages are processed per day at LinkedIn [28, 38, 39]. With highly-scalable distributed systems [27], one can surpass the limitations of a single machine in meeting the increasing demand in computation and storage. To accommodate growth, distributed systems employ two fundamental scaling strategies: *scale-out* (horizontal scaling), which adds more nodes to the cluster, and *scale-up* (vertical scaling), which increases resources per node. While these systems are inherently designed to scale, their complexity makes them prone to *scalability faults*.

Scalability faults are defined as faults that manifest only when one or more system aspects (e.g., number of nodes, data volume, concurrent requests) increase beyond certain thresholds, but remain latent at smaller scales [58]. Scalability faults are orthogonal to traditional fault categories such as

```
private void logDecommissioningNodesStatus() {
-   StringBuilder sb = new StringBuilder();
   for (DecommissioningNodeContext d : decomNodes.values()) {
+       StringBuilder sb = new StringBuilder();
       sb.append( /* decommissioning node status */ );
       /* ... */
       /* out-of-memory when size of decomNodes is large */
+       LOG.debug("Decommissioning node: " + sb.toString());
   }
-   LOG.info("Decommissioning Nodes: " + sb.toString());
}
```

Fig. 1: YR-6188 [12]: “... and yes, it does throw an OOM exception in case of large clusters”

functional, performance, security, or concurrency faults. A scalability fault may exhibit symptoms similar to these categories (e.g., performance degradation, resource exhaustion, or race conditions), but is fundamentally characterized by its *scale-dependent manifestation*: the fault remains hidden during small-scale testing and only emerges under large-scale conditions. Revealing scalability faults requires large-scale deployment, which is difficult and expensive to achieve during the development and testing phases. As a result, scalability faults are often discovered in production environments [14, 15, 16, 30, 33, 34], leading to resource shortage, performance degradation, or even system crashes.

Figure 1 shows an example of a scalability fault and its fix in Hadoop Yarn. In YR-6188 [12], developers notice that the unbounded design of a data structure and a large number (i.e.,  $\geq 2000$ ) of decommissioning nodes lead to a system crash due to an out-of-memory (OOM) error. The root cause is the combination of (1) the `for` loop whose number of iterations grows with the number of decommissioning nodes, and (2) an unbounded data structure `StringBuilder` whose size grows in every iteration of the loop. If the number of decommissioning nodes is large, the `StringBuilder` object will exhaust memory and trigger an OOM error. The fix is to move the `StringBuilder` object inside the `for` loop to leave it out of scope after each iteration, thus allowing the garbage collector to free up memory. This example illustrates the scale-dependent nature of scalability faults: the code functions correctly with a small number of decommissioning nodes but fails catastrophically when that number exceeds a threshold.

Unfortunately, scalability faults are not well understood. Previous work [58] has revealed their existence and provided a high-level overview of their potential impacts based on a small number of observations. However, many questions remain unanswered: Which aspects of a system cause scalability

\*Contributed equally. H.-N. Zhu, G. Ayinmode, and C. Rubio-González are with the University of California, Davis, United States (e-mail: hnzhu@ucdavis.edu; goayinmode@ucdavis.edu; crubio@ucdavis.edu). C. A. Stuardo and H. S. Gunawi are with the University of Chicago, United States (e-mail: cesar.stuardo@bytedance.com; haryadi@cs.uchicago.edu).

TABLE I: Breakdown of Faults in the Empirical Study.

| Dimension    | Examples                              | Cassandra | Hadoop | HBase | HDFS | Ignite | Kafka | MapReduce | Spark | Storm | Yarn | Total      |
|--------------|---------------------------------------|-----------|--------|-------|------|--------|-------|-----------|-------|-------|------|------------|
| Load         | # requests, RPCs, jobs                | 9         | 15     | 13    | 36   | 13     | 9     | 22        | 15    | 11    | 11   | 154        |
| Data         | # tables, files, dirs, or their sizes | 29        | 10     | 12    | 49   | 14     | 21    | 7         | 27    | 1     | 6    | 176        |
| Cluster      | # peers, datanodes, namenodes         | 29        | 5      | 9     | 17   | 17     | 14    | 2         | 1     | 2     | 2    | 98         |
| Failure      | # node failures                       | 0         | 2      | 2     | 3    | 1      | 1     | 4         | 1     | 1     | 1    | 16         |
| <i>Total</i> |                                       | 67        | 32     | 36    | 105  | 45     | 45    | 35        | 44    | 15    | 20   | <b>444</b> |

faults when scaled? Are there common root causes and/or code anti-patterns that induce scalability faults? Is it possible to systematically detect scalability faults without large-scale deployment? Each of these questions implies significant challenges, as scalability faults are often difficult to reproduce and diagnose unless the system is deployed at large scale.

To address the above questions, we conduct a comprehensive empirical study encompassing 444 scalability faults from 10 large-scale distributed systems. For each scalability fault, we manually inspect the issue report to identify (1) which system aspects are being scaled, and (2) the root causes and/or common anti-patterns that may induce scalability faults.

From (1), we report the aspects of the system that are subject to change in scale and significantly impact the system's scalability with their growth. We refer to such aspects as **scalable system dimensions** [50, 51], which we categorize into four kinds: (i) *load*, the volume of work the system processes (e.g., #requests, RPCs, jobs); (ii) *data*, the volume of items the system stores (e.g., #tables, files, directories, or their sizes); (iii) *cluster*, the size of the deployment (e.g., #peers, datanodes, namenodes); and (iv) *failure*, the number of concurrent component failures the system must tolerate (e.g., simultaneously failing nodes during recovery). Table I lists examples of each scalable system dimension and their corresponding number of faults.

From (2), we identify four root-cause categories of scalability faults: (i) *compute*, execution-time overhead tied to scale; (ii) *unbound*, resource-consumption overhead tied to scale; (iii) *bloat*, scalability-limiting data structure designs; and (iv) *logic*, logic flaws that surface only at large scale. Within these categories, we further identify 11 **anti-patterns**, i.e., concrete code or design forms that repeatedly induce scalability faults, with two or three anti-patterns per category, described individually in Section II. Across these anti-patterns, we observe that **dimensional code fragments (DCFs)** play a fundamental role. A DCF is an iterative code fragment whose number of iterations is correlated with one or more scalable system dimensions. For example, the `for` loop in Figure 1 is a DCF whose iteration count grows with the number of decommissioned nodes. DCFs are common and often necessary in scalable systems, and are neither faults nor anti-patterns by themselves; they become fault-relevant only when associated with certain anti-patterns (e.g., the unbounded `StringBuilder` in Figure 1). In our study, DCFs underlie 67.34% of scalability faults in the *compute* and *unbound* categories; the remaining 32.66% in the *bloat* and *logic* categories involve other mechanisms.

Then, by formalizing the insights from our empirical study, we design and implement SCALENS, a DCF-centered ap-

proach to detect scalability faults and identify their root causes while utilizing a single machine. SCALENS consists of two main components: SCALEVIEW and SCALEPICK, which perform dynamic and static analyses, respectively. SCALEVIEW uses runtime instrumentation together with scaling workloads to collect execution traces. These traces are then automatically analyzed to determine correlations between scalable system dimensions and code fragments to identify DCFs and categorize their computational complexity. SCALEPICK performs call-graph analysis on the DCFs to identify anti-patterns and pinpoint fragments that lead to scalability faults.

Our evaluation of SCALENS is two-fold. First, we evaluate SCALENS on 55 previously known real-world scalability faults from Cassandra, HDFS, and Ignite, which are widely-used distributed systems. Our evaluation shows that SCALENS is successful at fully detecting 36 (and partially detecting 2 more) out of 55 scalability faults, outperforming the baseline by 4.2 $\times$ . Second, we evaluate the capability of SCALENS to detect previously unknown scalability faults. Specifically, we use SCALENS to analyze the latest stable versions of Cassandra, HDFS, and Ignite, in which it detects a total of 334 DCFs with associated anti-patterns. After manual inspection, we find that all cases constitute problematic behaviors. We are in the process of reporting the new faults; so far 5 of reported DCFs have been confirmed by developers, and 4 more are under investigation.

In summary, this paper makes the following contributions:

- We conduct the largest-to-date study on scalability faults, from which we derive the existence of DCFs and conclude common scalability anti-patterns (Section II).
- We design and implement SCALENS, a novel approach that combines dynamic and static analyses to detect scalability faults based on the synergy between DCFs and anti-patterns (Section III).
- We evaluate SCALENS on 55 previously known real-world scalability faults found in Cassandra, HDFS, and Ignite, showing the effectiveness of SCALENS in detecting and precisely identifying root causes for 36 of them. Compared to the baseline, SCALENS improves the DCF detection by 4.2 $\times$  (Section IV-B).
- We demonstrate the ability of SCALENS to identify 334 DCFs with associated anti-patterns in the latest stable versions of Cassandra, HDFS, and Ignite. We discuss their behavior and implications, as well as our ongoing process of reporting new faults (Section IV-C).

## II. EMPIRICAL STUDY OF SCALABILITY FAULTS

This section presents our empirical study on scalability faults, with a discussion of their root-cause categories and

common anti-patterns. We describe the methodology of our empirical study below.

1) *System Selection*: We considered 10 large-scale distributed systems: Cassandra [17] (CA), Hadoop [18] (HA), HBase [19] (HB), HDFS [20] (HD), Ignite [21] (IG), Kafka [22] (KF), MapReduce [23] (MR), Spark [24] (SP), Storm [25] (ST), and Yarn [26] (YR). We chose these systems because (1) they are open-source with public and highly organized JIRA issue tracking systems, (2) they are typically deployed at scale and (3) the selection covers the most common categories of distributed systems, including databases, file systems, and parallel computing.

2) *Issue Selection*: We collected all (over 199K) issue reports from the JIRA issue tracking systems of the target systems from 2007 to 2024. Our initial attempt to use keywords to automatically identify reports describing scalability faults was unfruitful due to the wide variety of scalability problems and the lack of specific keywords (detailed in Section V-B). Therefore, we performed a two-pass manual inspection. In the first pass, we read the title and description of the reports to identify bug reports that involve the increase in the scale of any aspect of the system. In the second pass, we performed a more detailed review of the candidate bug reports by examining the aspects being scaled when symptoms are observed. We then categorized those into four dimensions: *load*, *data*, *cluster*, and *failure*. This process involved 8 individuals and spanned over one year, leading to the identification of 444 scalability faults, detailed in Table I.

3) *Root-Cause and Anti-Pattern Analysis*: We manually analyzed the 444 scalability faults to understand the nature of each fault and its root cause. This process consisted of examining the contents of the report, discussions, source code, and available patches/pull requests. To ensure reliability and minimize bias, we employed a multi-stage adjudication process. First, each issue was independently reviewed and tagged by two authors, who categorized it by root-cause category (compute, unbound, bloat, or logic) and its associated anti-pattern. Second, a third author reviewed all cases where the initial taggers disagreed or expressed uncertainty. Finally, all authors participated in group discussions to resolve remaining conflicts and validate the final categorization. All analyzed issue reports are publicly available in the respective JIRA repositories, and we provide supplementary material including the complete set of bug reports analyzed in this study, detailed tagging guidelines, and a full catalog of issue categorizations to support reproducibility and enable further research.

Our analysis has revealed four **root-cause categories** of scalability faults, *compute*, *unbound*, *bloat*, and *logic*, and 11 **anti-patterns** grouped under them, as shown in Figure 2. Across these anti-patterns, a recurring structural property stands out: many of them involve code fragments that iterate (either implicitly or explicitly) a number of times that grows with the scale of the system. We refer to these as **dimensional code fragments (DCFs)**. DCFs are not themselves part of the taxonomy; instead, they are a cross-cutting property shared mainly by the *compute* and *unbound* anti-patterns. Below, we describe the root-cause categories and anti-patterns in turn, drawing on DCFs where the anti-pattern's behavior is directly

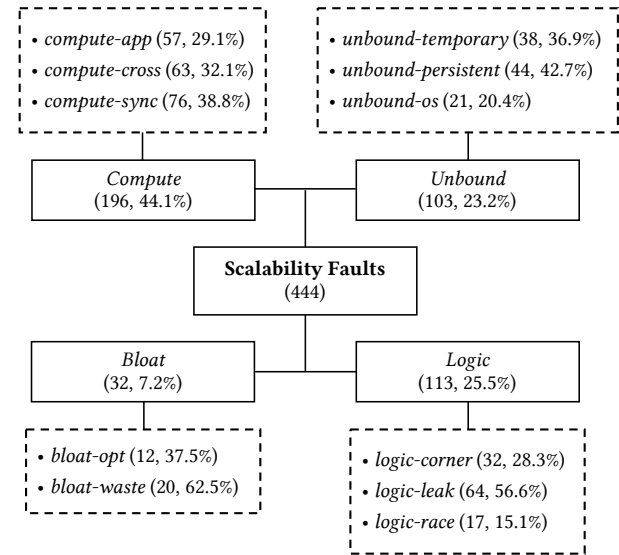


Fig. 2: Scalability Fault Anti-Patterns.

tied to scale-dependent iteration.

#### A. Compute Faults

*Compute* faults are related to bottlenecks caused by the increase in size of one or more scalable system dimensions (e.g., *#partitions* in Figure 3a). It is the largest category with **196** fault reports, representing **44.1%** of the total, and includes 3 anti-patterns:

- **compute-app** faults, with **57** reports, account for **29.1%** of this category and represent cases in which bottlenecks are caused by DCFs in performance critical paths. Even when the computations inside the DCF are not considered costly, the complexity (commonly  $\geq 2$ ) produces notable performance degradation. For example, in **IG-8681** [9] (Figure 3a), `unwindEvicts` is invoked every time a remote Ignite command is processed. This method contains a quadratic DCF of  $O(P^2, P = \#partitions)$  (highlighted at lines 3 and 5). Since remote command execution is part of the critical path, invoking this method is reportedly problematic.

- **compute-cross** faults, with **63** reports, account for **32.1%** of this category and represent cases in which bottlenecks are caused by DCFs containing cross-system (e.g., methods from pluggable components, external clients, or IO operations) method calls. When cross-system method calls are performed inside DCFs, their computational cost gets amplified [29, 83] by the size of the related dimension, reportedly leading to severe performance degradation. For example, in **KF-5642** [10] (Figure 3b), `updateOwner` contains a linear DCF that iterates over each partition to update the ownership for a Kafka topic. Each iteration triggers synchronous communication with a cross-layer component (ZooKeeper, highlighted at line 5). As the number of partitions grows, the number of synchronous messages increases and the execution time of the whole operation becomes dependent on the network speed and the load on the external component.

- **compute-sync** faults, with **76** reports, account for **38.8%** of this category and represent cases in which synchronization

|                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                              |                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1 void unwindEvicts(Partition[] all){ 2   // for every partition 3   for(Partition o : all){ 4     // for every version 5     for(Partition i : o.versions()){ 6       if(i.isExpired()) { 7         i.markForRemoval(); 8       } 9     } 10  } 11 } </pre> | <pre> 1 void updateOwner(Partition[] ps){ 2   // for every partition 3   for(Partition p : ps){ 4     // cross-system call 5     ZKClient.send(p.id, p.owner); 6   } 7 } </pre>                              | <pre> 1 void updateTokens(Token[] ts){ 2   // holds a lock 3   writeLock(); 4   // for every token 5   for(Token t : ts){ 6     if(!cachedTokens.contains(t)){ 7       cachedTokens.add(t); 8     } 9   } 10  writeUnlock(); // release 11 } </pre> | <pre> 1 void loadCache(Domain[] ds){ 2   for(Domain d : ds){ 3     // bring all into memory 4     Entity[] es = d.load(); 5     for(Entity e : es){ 6       // accumulate in cache 7       cache.put(e); 8     } 9   } 10 } </pre> |
| (a) compute-app                                                                                                                                                                                                                                                    | (b) compute-cross                                                                                                                                                                                            | (c) compute-sync                                                                                                                                                                                                                                    | (d) unbound-temporary                                                                                                                                                                                                              |
| <pre> 1 void processRequest(Request r){ 2   // execute 3   Result re = r.execute(); 4   if(re.isDone()){ 5     // add without checking 6     // size or memory usage 7     responseQueue.add(re); 8   } 9 } </pre>                                                 | <pre> 1 void processRequest(Request r){ 2   Result re = r.execute(); 3   // one connection per request 4   Socket connection = \ 5     new Socket(r.ip, r.port, 10); 6   replyTo(connection, rs); 7 } </pre> | <pre> 1 class IndexSummary { 2   // Long[] is preferable 3   List&lt;Long&gt; pos = new ArrayList&lt;&gt;(); 4   // byte[] is preferable 5   Map&lt;Byte, Byte&gt; keys = new Map&lt;&gt;(); 6   // ... 7 } </pre>                                  | <pre> 1 class StatsMetadata { 2   // heavy object 3   byte[] min = new byte [1000000]; 4   byte[] max = new byte [1000000]; 5   // ... 6 } </pre>                                                                                  |
| (e) unbound-persistent                                                                                                                                                                                                                                             | (f) unbound-os                                                                                                                                                                                               | (g) bloat-opt                                                                                                                                                                                                                                       | (h) bloat-waste                                                                                                                                                                                                                    |

Fig. 3: Code Samples for the *Compute*, *Unbound*, and *Bloat* Faults. From (a) to (h), they are based on IG-8681, KF-5642, CA-5456, YR-7147, CA-15013, HA-15696, CA-5506, and CA-15400, respectively.

bottlenecks are caused by DCFs protected by global locks. Such fault anti-patterns have been observed previously [62, 77] in centralized architectures such as HDFS and include both cases in which the locks wrap DCFs and cases in which the locks are held inside the DCF, the former being the most frequent. For example, in CA-5456 [3] (Figure 3c), the method `updateTokens` is invoked every time *range movements* are performed to maintain consistency when a Cassandra cluster's topology is changing. The operation contains a linear DCF (line 4) and is performed while holding a global lock (highlighted at line 2). Other threads, in particular the ones that handle membership changes, have to wait until the whole operation finishes. As the number of tokens grows, the execution time of the code block between lines 4 and 8 grows too, reportedly causing a negative impact on elasticity.

## B. Unbound Faults

*Unbound* faults are related to unconstrained resource consumption caused by the increase in size of one or more scalable system dimensions (e.g.,  $\#entities/domains$  in Figure 3d). This category, with 103 fault reports, represents 23.2% of the total and includes 3 anti-patterns:

- **unbound-temporary** faults, with 38 reports, account for 36.9% of this category and represent cases in which temporary memory allocations, such as stack-level data structures, grow without bounds in response to the growth of one or more dimensions, reportedly causing out-of-memory errors and performance degradation due to frequent garbage collection. This anti-pattern can be intuitively related to the lack of buffering or paging when loading data from external sources (e.g., files or databases) [32, 48]. For example, in YR-7147 [13] (Figure 3d), the method `loadCache` is invoked every time the component *Timeline Server* [40] of Yarn is started and contains a quadratic DCF of  $O(D * E, D = \# domains, E = \# entities)$ . The cache is loaded from disk (highlighted at line 7), but the whole file is materialized in memory in one method call (highlighted at line 4), causing the component to run out of memory when the

related files get too big. As reported, the component is killed and it can be restarted only if there is more memory available or the caching feature is disabled.

- **unbound-persistent** faults, with 44 reports, account for 42.7% of this category and represent cases in which long-lived data structures, such as inbound/outbound message processing queues or multi-purpose caches, grow without bounds in response to the growth of one or more dimensions, reportedly causing out-of-memory errors. Note that the difference between this and the previous category is related to object lifetime, purpose and expected functionality: While the previous refers to temporary allocations typically used as containers/helpers, this category refers to persistent allocations that, accompanied by the necessary logic, are supposed to accommodate the growth of one or more dimensions but fail to do so due to issues in said logic. For example, in CA-15013 [1] (Figure 3e), the Cassandra request processor contains a DCF of  $O(R, R = \# requests)$  that accumulates reportedly large responses in a persistent queue (highlighted at line 7). If the responding threads (which consume those objects) cannot keep up, the queue grows without bounds and the node runs out of memory.

- **unbound-os** faults, with 21 issues, account for 20.4% of this category and represent cases in which OS resource allocations, such as threads and file descriptors, grow without bounds in response to the growth of one or more dimensions potentially exhausting those resources. These faults are uncommon in newer versions since the unbounded increase of threads and/or sockets is a known issue and is typically targeted in early fixes. For example, in HA-15696 [5] (Figure 3f), an encryption-related HDFS component contains a DCF of  $O(R, R = \# requests)$  that creates a single `Socket` (with a corresponding file descriptor) every time a request is processed. Even if these connections are short-lived, the default idle-timeout is set to 10 seconds (highlighted at line 5). As reported, an explosive increase in the number of requests ends up in failures (the system runs out of file descriptors).

### C. Bloat Faults

*Bloat* faults are related to data structures with scalability-limiting design [64, 65, 76], causing unexpected increases in memory consumption at larger scales. This category, with **32** fault reports, represents **7.2%** of the total and includes 2 anti-patterns:

- **bloat-opt** faults, with **12** issues, account for **37.5%** of this category and represent cases in which data structure design includes “space-time trade-offs” [49, 88] that, albeit intended to improve performance, obviate memory constraints at larger scales. For example, in **CA-5506** [4] (Figure 3g), Cassandra nodes utilize a data structure called `IndexSummary` that uses the collections framework [36] to store internal indexes. As the number of live `IndexSummary` instances grows, a  $2\times$  per-element overhead in positions (highlighted at line 3) and a  $10\times$  per-element overhead in keys causes 70% memory bloat in a cluster with billions of rows.

- **bloat-waste** faults, with **20** reports, account for **62.5%** of this category and represent cases in which data structure design includes unnecessarily large preallocated fields (e.g., fixed arrays or buffers), that obviate memory constraints at larger scales. Note that the difference between this and the previous category is related to functionality: The previous focuses on allocations related to more complex techniques (such as indexing) which require a much more fine-grained design, while this category focuses on unnecessarily large (and often unused) space being preallocated (i.e., *wasted*). For example, in **CA-15400** [2] (Figure 3h), Cassandra creates one `StatsMetadata` instance for each `BigTableReader`, an internal data structure whose size is correlated to the load dimension. The former declares two fixed-size 1MB buffers (highlighted at lines 3 and 4) that, according to developers, are frequently not used in their entirety. When under high load, many live `BigTableReader` instances lead to high memory usage and eventually end in out-of-memory errors, affecting not one but several nodes in the cluster.

### D. Logic Faults

*Logic* faults are related to logic issues that become visible at large scales, such as corner cases, memory or resource leaks, and data races. This category, with **113** fault reports, represents a surprising **25.5%** of the total and includes 3 anti-patterns:

- **logic-corner** faults, with **32** issues, account for **28.3%** of this category and represent cases in which faulty logic (e.g., wrong choice of datatypes) is problematic at larger scales but harmless at smaller scales. For example, in **ST-3256** [11], a large Storm topology is used to communicate with custom terminals in a client-server fashion. The communication is handled by *worker* threads, which are uniquely identified across the whole deployment using a numeric 16-bit unsigned integer. Due to this, no more than 32K workers can be instantiated concurrently, which imposes a severe limit to parallelism (in the reporter’s words, “... is a disaster for large scale computing”).

- **logic-leak** faults, with **64** issues, account for **56.6%** of this category and represent cases in which memory, OS resources, or storage leaks at larger scales. For example, in **HD-**

**13039** [7], HDFS *erasure coding* component exhibits a classic OS (file descriptor) leak: `createReader` is invoked when creating a `Reader` object on a per-request basis. However, a connection is created but never closed in the case of file creation failures. This leads to extended downtimes (no more file descriptors available) when many of these failures happen.

- **logic-race** faults, with **17** issues, account for **15.1%** of this category and represent cases in which race conditions are observed at larger scales (e.g., when a large number of peers join the cluster at the same time), causing effects that range from data inconsistencies to system failures. For example, in **HA-16385** [6], a 53K-container heavily loaded Hadoop HDFS cluster is dealing with multiple node failures at the same time. In the middle of this failure storm, a race condition related to the calculation of the number of live peers triggers an assertion failure that ends up killing the HDFS *namenode*.

### E. Key Insights

Our study has confirmed that scalability faults are challenging to detect. We identified four main categories of scalability faults, from which *compute* and *unbound* stand out; more than half of all faults (67.34%) fall into these categories, which are caused by the combination of DCFs and anti-patterns. From them, we observe the following scalability anti-patterns:

- *compute-app*: DCFs placed in performance-critical paths.
- *compute-cross*: DCFs with API or IO operations.
- *compute-sync*: DCFs protected by global locks.
- *unbound-temporary*: DCFs with memory allocations.
- *unbound-persistent*: DCFs that grow the size of objects.
- *unbound-os*: DCFs that consume OS resources.

Based on these observations, identifying DCFs and associated anti-patterns is essential to detect scalability faults. We also observe that:

- **The nature of DCFs** can be **explicit** or **implicit**. Explicit DCFs are `for` or `while` loops directly visible in the source code of the system or its related libraries, as shown in Figures 3a-d, and implicit DCFs refer to cases where no loops are directly present, but the number of method invocations increases with one or more system dimensions as shown in Figures 3e-f. Due to the nature of DCFs, especially implicit ones, they are difficult to detect without runtime information.

- **DCFs are not necessarily problematic by themselves**, as they represent basic building blocks of distributed systems. For example, for *compute*-related DCFs, we observe that they become problematic only when present in performance sensitive paths, make external API or IO calls, or are protected by global locks. Therefore, detecting DCFs alone is not sufficient and identifying associated anti-patterns is essential to accurately detect scalability faults.

- **DCF complexity matters**, but it is not a strong indicator of faults by itself. For example, high-complexity functions could be considered acceptable in background paths but problematic in foreground or user-facing paths.

In addition to *compute* and *unbound* faults, we also observe *bloat* and *logic* faults. *Bloat* faults are related to inefficient data structure designs that lead to unnecessary memory overhead as the system scales, while *logic* faults are logic issues that

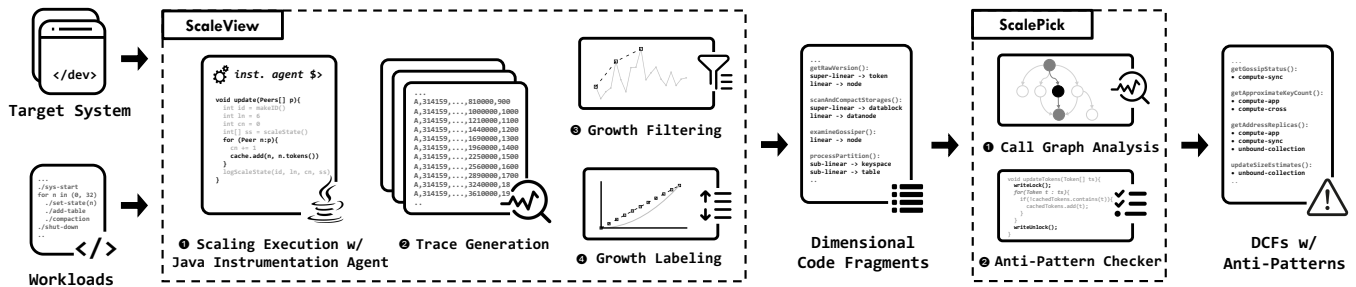


Fig. 4: SCALELENS Workflow.

become visible when the system is deployed at larger scales. Unlike *compute* and *unbound* faults, these two categories do not involve DCFs, and may require memory footprint and concurrency analyses for their detection. Together, they account for a smaller portion of the scalability faults (32.66%).

In the next section, we describe our approach, SCALELENS, to detect *compute* and *unbound* scalability faults. We leave further analysis of *bloat* and *logic* faults as future work.

### III. OUR APPROACH: SCALELENS

We present SCALELENS, an approach for detecting scalability faults in large-scale distributed systems. SCALELENS has two major components, illustrated in Figure 4: (1) SCALEVIEW that performs *dynamic* analysis to identify DCFs, and (2) SCALEPICK that performs *static* analysis to detect anti-patterns associated with DCFs.

#### A. SCALEVIEW

The goal of SCALEVIEW is to list all DCFs for a given system and their relationships with the system dimensions. Such information can help developers to *view* all potential scalability bottlenecks from a scaling perspective prior to actual production deployment.

Given the source code of a target system and a set of scaling workloads, SCALEVIEW (1) instruments the code to obtain execution traces, (2) parses and analyzes the execution traces to discover execution trends, (3) filters out noise to identify real growth, and (4) labels the DCFs according to their relationships to the system dimensions.

1) *Scaling Workloads and System Dimensions*: The first step is to write a **scaling workload**: a script that systematically increases one or more scalable system dimensions (the *scalable* aspects of interest) while invoking the target system's APIs. This process is manual and requires familiarity with deployment and operational practices of the target system. It is a one-time effort and can be reused across different versions of the system (implications are discussed in Section V-C).

Figure 5 shows an example workload scaling the number of tables in a Cassandra cluster. The maximum number of tables is set on line 3, and the `for` loop on line 4 adds one table at a time. On lines 6 and 7, a table is created and data is inserted via `cqlsh`, the database operation interface provided by Cassandra. On line 8, compaction is run via `nodetool`, the system management utility provided by Cassandra. Compaction is optional, and can be replaced with other system

```
1 # Cassandra: scaling up #tables and run compaction
2 ./setup-node && ./start-node
3 MAX_TABLES=128
4 for (( i = 1; i <= $MAX_TABLES; i++ )); do
5     ./set-scale-state $i
6     ./cqlsh -e "CREATE TABLE (...)" # create a new table
7     ./cqlsh -e "INSERT INTO (...)" # insert dummy data
8     ./nodetool (...) compact (...) # run compaction
9     ./sleep
10 done
11 ./shut-down
```

Fig. 5: Scaling Workload Example.

operations (e.g., snapshot, scrub, repair, etc.) based on the interest of the user of SCALEVIEW.

The function `set-scale-state` on line 5 of Figure 5 is an API provided by SCALEVIEW for reporting the current **scale state** of the workload: the numeric value(s) of the dimension(s) being scaled at that moment of execution (here, the current number of tables, e.g., 17 after the 17th loop iteration). SCALEVIEW uses this reported value to correlate each observed iteration count of a code fragment with the scale of the target dimensions. A multi-dimensional scale state is also supported: for example, a workload that scales tables and rows simultaneously would call `set-scale-state` with a pair of values, such as (17, 10000) once the 17th table contains 10000 rows.

2) *Runtime Instrumentation*: After scaling workloads are triggered, SCALEVIEW monitors the relationship between the current size of the dimensions and the number of iterations performed by a code fragment. Such monitoring is done by using bytecode-level runtime instrumentation [35, 37]. Based on the insights from Section II, SCALEVIEW considers both explicit and implicit iterations.

• **Explicit**. As shown in Figure 6a, once a loop is detected by SCALEVIEW in the source code of the target system, SCALEVIEW adds a unique ID, line number, and a counter variable to the loop. Library loops are also instrumented but in this case the unique ID corresponds to the ID of the caller, as shown in Figure 6b. This is done to maintain the scope of reporting within the target system. Finally, for both types of loops, SCALEVIEW retrieves the scale state information with `getScaleState()` and logs the scale state along with the counter variables with the `logScaleState()`.

• **Implicit**. SCALEVIEW captures implicit iterations by capturing the relationship between the number of invocations of each method and the current state of the scaling dimensions.

```

1 void update(Peers[] p) {
2   int id = makeId();
3   int ln = 5, cn = 0;
4   int[] ss = getScaleState();
5   for(Peer n : p) {
6     cn += 1;
7     cache.add(n, n.tokens());
8   }
9   logScaleState(id, ln, cn, ss);
10 }
11
12 void add(K k, V[] v) {
13   int id = callerId();
14   int ln = callerLine();
15   int cn = 0;
16   int[] ss = getScaleState();
17   for(V val : v) {
18     cn += 1;
19     map.put(k, val);
20   }
21   logScaleState(id, ln, cn, ss);
22 }

```

(a) Application (b) Library

Fig. 6: Runtime Instrumentation. Code in grey is instrumented.

3) *Trace Generation*: The combination of scaling workloads and instrumentation produces traces that capture runtime information about loop executions. Specifically, a tuple  $\langle type, methodID, line\#, execID, \#iterations, SS \rangle$  is recorded for each loop execution. In the tuple,  $type \in \{A, L, I\}$  represents the type of the loop: application, library, or implicit.  $methodID$  and  $line\#$  record the method and location of the loop, respectively.  $execID$  is a unique identifier for each execution of the loop, and  $\#iterations$  is the final iteration count of a given loop execution.  $SS$  represents the scale state in an array with one or more numbers for one- or multi-dimensional executions, respectively. For example, tuple  $\langle A, 314159, 64, 265358, 810000, 900 \rangle$  describes the loop execution with ID 265358: an execution of the application loop located in the method with ID 314159 on line 64 iterated a total of 810000 times when the number of nodes (i.e., the scale state) reached 900, respectively.

4) *Growth Filtering*: The goal of growth filtering is to detect DCFs by identifying iterative code fragments with a positive correlation between the number of iterations and the target scaling dimensions, and discarding the rest. This is done with heuristics based on the growth patterns shown in Figure 7. Besides the trivial clear flat (Figure 7a) and clear growth (Figure 7b) patterns, we also observed noisy growth (Figure 7c) and noisy flat (Figure 7d) patterns. SCALEVIEW performs a 3-step empirical-driven filtering process: (1) only keep the data points that are larger than the maximum of all the previous data points (i.e., the dashed boxes in Figure 7), (2) remove the code fragments with less than 10% of data points retained after the first step (e.g., Figure 7a and Figure 7d), and (3) keep the code fragments whose correlation to the scaling dimension is high (i.e.,  $\geq 0.9$ ). Such a filtering process excludes both clear and noisy flat patterns.

5) *Growth Labeling*: Finally, SCALEVIEW categorizes DCFs based on their previously filtered growth trends as (1) super-linear, (2) linear, or (3) sub-linear. To do so, we select a set of theoretical complexity models (e.g.,  $I = O(D^2)$  is super-linear,  $I = O(D)$  for linear, and  $I = O(\log D)$  for sub-linear, where  $I$  is the number of iterations and  $D$  is the scale of dimension) and use similarity measures [43] to detect the closest theoretical model and assign the corresponding label to the fragment. We consider that this simple labeling can be useful for developers to estimate the urgency or priority of the issue. For example, a super-linear (e.g.,  $O(N^2)$  or above) code fragment intuitively brings more negative impacts than a linear (e.g.,  $O(N)$ ) fragment under the same condition.

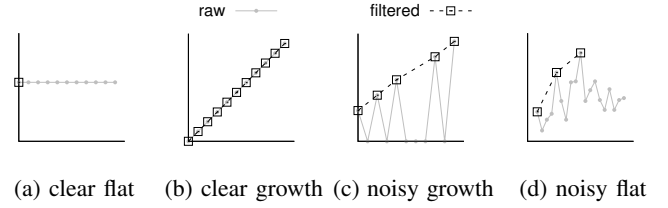


Fig. 7: Growth Patterns.

## B. SCALEPICK

DCFs reported by SCALEVIEW represent potential scalability bottlenecks in the system, but as discussed previously, not all DCFs are necessarily problematic. Thus, given the source code of a target system, as well as the DCFs found by SCALEVIEW, SCALEPICK (1) performs call graph analysis from DCFs, and (2) uses static checkers to identify associated anti-patterns, if any.

SCALEPICK incorporates a set of simple static checkers that identify the first 6 anti-patterns from our empirical study using CODEQL [31], and all checkers are applied to each DCF to flag associated anti-patterns. SCALEPICK's checkers use **program slicing** [45], a technique that computes the set of statements that may influence (backward slice) or be influenced by (forward slice) a given statement. We apply slicing over the call graph of the target system: a backward slice from a DCF gathers the methods that can reach it, while a forward slice from a DCF gathers the methods and the allocation, I/O, or synchronization sites it may reach. Each checker below inspects a specific slice and flags the DCF if the slice contains a checker-specific sink. Details of the checkers are discussed below.

- The **compute-app** checker detects DCFs in performance or operational critical paths. Those code paths could be user-facing or foreground depending on the testing requirements. To detect this anti-pattern, we perform backward slicing [45] from the DCFs over the call graph to check reachability from any specified user-facing or foreground API calls. Reachable DCFs are flagged as *compute-app*.

- The **compute-cross** checker finds external API or IO operations within DCFs. Such operations could be read/write interactions with disk, network, or database, which are often performed in a synchronous fashion. To detect this anti-pattern, we perform forward slicing [45] from the DCFs over the call graph to find DCFs associated with external API or IO invocations. Such DCFs are then flagged as *compute-cross*.

- The **compute-sync** checker finds instances where lock contentions may be caused by DCFs holding a global lock. Such lock contentions typically happen in foreground/background interactions. To detect this anti-pattern, we perform both backward and forward slicing from the DCFs over the call graph. If a DCF holds a global lock that is also used by other components, then it is flagged as *compute-sync*.

- The **unbound-temporary** checker finds unbounded memory allocations whose lifetime is local to the method that contains the DCF. We compute the forward slice from the DCF and check whether the slice contains a memory-growth sink on a *method-local* target, i.e., an object scoped to the method: (i)

TABLE II: Evaluation Results.  $T_{DCF}$  denotes the type of reported DCF, where  $E$  stands for explicit and  $I$  for implicit. In  $DCF$  columns for both SCALENS and SFIND,  $\times$  denotes that the related DCF was not detected and  $\checkmark$  denotes positive results. In *Anti-Pattern* column under SCALENS,  $\times$  denotes the anti-pattern is not detected given the detection of DCF, and empty cell means not evaluated because the DCF is not detected.

| Issue Information |           |                 | SCALENS      |              | SFIND    | Issue Information |           |                 | SCALENS      |              | SFIND        | Issue Information |           |                 | SCALENS      |              | SFIND        |
|-------------------|-----------|-----------------|--------------|--------------|----------|-------------------|-----------|-----------------|--------------|--------------|--------------|-------------------|-----------|-----------------|--------------|--------------|--------------|
| ID                | $T_{DCF}$ | Anti-Pattern    | DCF          | Anti-Pattern | DCF      | ID                | $T_{DCF}$ | Anti-Pattern    | DCF          | Anti-Pattern | DCF          | ID                | $T_{DCF}$ | Anti-Pattern    | DCF          | Anti-Pattern | DCF          |
| CA-19534          | $I$       | $u$ -persistent | $\times$     |              | $\times$ | CA-14660          | $E \& I$  | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$     | HD-16100          | $E$       | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\checkmark$ |
| CA-19477          | $E \& I$  | $c$ -cross      | $\checkmark$ | $\checkmark$ | $\times$ | CA-14239          | $E$       | $u$ -persistent | $\checkmark$ | $\checkmark$ | $\times$     | HD-15621          | $I$       | $u$ -persistent | $\times$     |              | $\times$     |
| CA-19412          | $E \& I$  | $u$ -persistent | $\checkmark$ | $\checkmark$ | $\times$ | CA-14096          | $E$       | $u$ -persistent | $\checkmark$ | $\checkmark$ | $\times$     | HD-15406          | $E$       | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$     |
| CA-19336          | $I$       | $u$ -temporary  | $\checkmark$ | $\checkmark$ | $\times$ | CA-13993          | $I$       | $c$ -cross      | $\checkmark$ | $\checkmark$ | $\times$     | HD-14859          | $E$       | $c$ -app        | $\checkmark$ | $\checkmark$ | $\checkmark$ |
| CA-19107          | $E \& I$  | $c$ -app        | $\checkmark$ | $\checkmark$ | $\times$ | CA-13948          | $E$       | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$     | HD-14854          | $E$       | $c$ -sync       | $\times$     |              | $\times$     |
| CA-18773          | $E \& I$  | $c$ -app        | $\checkmark$ | $\checkmark$ | $\times$ | CA-13923          | $E \& I$  | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$     | HD-14771          | $E$       | $c$ -cross      | $\times$     |              | $\times$     |
| CA-18546          | $E \& I$  | $u$ -persistent | $\checkmark$ | $\checkmark$ | $\times$ | CA-13569          | $I$       | $u$ -persistent | $\times$     |              | $\times$     | HD-14657          | $I$       | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$     |
| CA-17787          | $I$       | $u$ -temporary  | $\checkmark$ | $\checkmark$ | $\times$ | CA-13299          | $I$       | $u$ -persistent | $\times$     |              | $\times$     | HD-14613          | $E$       | $c$ -app        | $\checkmark$ | $\checkmark$ | $\checkmark$ |
| CA-17691          | $E \& I$  | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$ | CA-13215          | $E \& I$  | $c$ -app        | $\checkmark$ | $\checkmark$ | $\times$     | HD-14370          | $E$       | $c$ -cross      | $\times$     |              | $\times$     |
| CA-17342          | $E \& I$  | $c$ -app        | $\checkmark$ | $\checkmark$ | $\times$ | CA-13065          | $E$       | $c$ -sync       | $\times$     |              | $\times$     | HD-14366          | $E$       | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\checkmark$ |
| CA-16380          | $E \& I$  | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$ | CA-12281          | $I$       | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$     | HD-14201          | $I$       | $c$ -cross      | $\times$     |              | $\times$     |
| CA-16261          | $I$       | $u$ -persistent | $\times$     |              | $\times$ | CA-12245          | $E$       | $c$ -sync       | $\times$     |              | $\times$     | HD-14171          | $E$       | $c$ -app        | $\checkmark$ | $\checkmark$ | $\checkmark$ |
| CA-16201          | $E$       | $u$ -temporary  | $\checkmark$ | $\checkmark$ | $\times$ | CA-11748          | $I$       | $u$ -persistent | $\times$     |              | $\times$     | HD-13821          | $I$       | $c$ -sync       | $\times$     |              | $\times$     |
| CA-15364          | $I$       | $c$ -app        | $\checkmark$ | $\checkmark$ | $\times$ | CA-10654          | $E$       | $c$ -sync       | $\times$     |              | $\times$     | HD-13768          | $E$       | $c$ -sync       | $\times$     |              | $\times$     |
| CA-15141          | $E \& I$  | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$ | CA-9882           | $E \& I$  | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\checkmark$ | HD-13692          | $E$       | $u$ -persistent | $\checkmark$ | $\checkmark$ | $\checkmark$ |
| CA-15013          | $I$       | $u$ -persistent | $\times$     |              | $\times$ | HD-17097          | $I$       | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$     | IG-14076          | $E \& I$  | $u$ -persistent | $\checkmark$ | $\checkmark$ | $\times$     |
| CA-14855          | $E \& I$  | $u$ -persistent | $\checkmark$ | $\checkmark$ | $\times$ | HD-17096          | $E$       | $u$ -temporary  | $\checkmark$ | $\times$     | $\checkmark$ | IG-12189          | $I$       | $c$ -cross      | $\checkmark$ | $\checkmark$ | $\times$     |
| CA-14840          | $I$       | $u$ -persistent | $\checkmark$ | $\times$     | $\times$ | HD-16989          | $I$       | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$     | IG-12087          | $I$       | $c$ -sync       | $\checkmark$ | $\checkmark$ | $\times$     |
| CA-14747          | $I$       | $u$ -persistent | $\times$     |              | $\times$ |                   |           |                 |              |              | Total        | 55                |           |                 | 38           | 36           | 9            |

a fresh allocation (e.g., a constructor call for a Java collection class such as ArrayList, HashMap, or StringBuilder; a Java array creation `new T[n]`; or an external buffer-returning API whose return value is not released in the slice), or (ii) a size-growing call (e.g., `append`, `add`, `put`) on a mutable container declared in the containing method (and hence unreachable after the method returns). If either holds, the DCF is flagged as *unbound-temporary*.

- The **unbound-persistent** checker detects unbounded growth of long-lived objects reachable from a DCF, i.e., objects whose lifetime outlives the enclosing method. We compute the forward slice from the DCF and check whether it contains a size-growing call on a *non-local* target: an instance field, a static field, or a mutable object passed by reference (e.g., a shared queue, map, or cache). If such a call is in the slice, the DCF is flagged as *unbound-persistent*.

- The **unbound-os** checker identifies DCFs that consume system resources, such as file descriptors, sockets, or threads, without proper management. Such resources are often limited and could be exhausted when the system scales. To detect this anti-pattern, we perform forward slicing from the DCFs over the call graph to see if the DCF consumes system resources. If so, the DCF is flagged as *unbound-os*.

To demonstrate these checkers in practice, consider the for-loop DCF from YR-6188 [12] (the motivating example from Section I). In this example, a `StringBuilder` declared within the method containing the DCF is updated via repeated calls to `append`, which is reachable from the DCF. Because the buffer's size grows with every iteration, the unbound-temporary checker identifies this as a "size-growth sink" under case (ii). This detection aligns with both the manual tagging of YR-6188 [12] and SCALEVIEW's runtime data, which shows the buffer growing linearly with the number of decommissioning nodes.

#### IV. EVALUATION OF SCALENS

This evaluation aims to answer the following questions:

**RQ1** How effective is SCALENS in identifying previously known real-world scalability faults, and how does it compare to the baseline?

**RQ2** Can SCALENS detect previously unknown scalability faults in the latest stable versions of distributed systems?

##### A. Experimental Setup

1) *Benchmark*: As far as we know, there is no existing benchmark that targets scalability faults in distributed systems. Hence, we build our own based on the scalability faults we collected for our empirical study (Section II). From the 444 scalability faults, we focus on those reported for Cassandra, HDFS, and Ignite, the three systems with the most scalability faults (217 combined, as shown in Table I) and the most up-to-date documentation. Then, we (1) determine if the system's versions related to the faults are deployable and runnable, and (2) select the faults that are within the scope of SCALENS, i.e., *compute* and *unbound* faults. The steps above result in 55 real-world scalability issues, as listed in Table II.

2) *Baseline*: The most closely related work to SCALENS is SCALECHECK [77], an approach designed to discover scalability faults. SCALECHECK has two components: SFIND and STEST. Among the two, SFIND is responsible for detection, exposing *scale-dependent loops* through program analysis. Such loops are a narrower form of DCFs. Since both SFIND and SCALEVIEW identify code fragments with scale-correlated iteration counts from source code and workloads, we use SFIND as the baseline for SCALEVIEW. Technical details of SCALECHECK and a head-to-head comparison with SCALENS are provided in Section VI.

3) *Workloads*: We implemented 8 workloads for Cassandra, 3 for HDFS, and 2 for Ignite. In those, we scaled a total of 8 dimensions (*nodes*, *tokens*, *tables*, *rows* for Cassandra, *datanodes*, *datablocks*, *files* for HDFS, and *rows* for Ignite) and exercised a total of 6 APIs (*snapshot*, *compaction*, *repair* for Cassandra, *snapshot*, *snapshotDiff* for HDFS, and *query* for Ignite). The dimensions and APIs are selected based on their

popularity, while SCALELENS users can select any dimension or API of interest. Each workload comprises 80–100 lines of code in shell scripts and is reutilized for every version we test. The same workloads are given to SFIND and SCALEVIEW.

4) *Environment*: All experiments are conducted on a single machine with 40-core Xeon Platinum 8380 CPU and 256 GB of RAM. All experiments, including the benchmark, are packaged into Docker images and are publicly available to ensure the reproducibility of the results.

### B. RQ1: Effectiveness and Comparison with Baseline

The goal of RQ1 is to evaluate the effectiveness of SCALELENS in detecting known scalability faults, and to provide a comparison with the baseline SFIND.

We evaluate SCALELENS on our benchmark of 55 real-world scalability faults drawn from our empirical study. For each fault, we examine whether SCALEVIEW detects the corresponding DCF, and whether SCALEPICK identifies the correct anti-pattern associated with that DCF. As summarized in Table II, SCALEVIEW detects the correct DCFs for 38 out of 55 faults, including both explicit and implicit iterations. For each detected DCF, SCALEVIEW also correctly identifies the associated system dimensions (not shown in Table II due to space constraints). We found that all 17 cases in which SCALEVIEW fails to detect DCFs are related to multi-threading. For example, HD-13768 [8] is a scalability fault where all threads are forced to wait for the slowest to complete, a scenario that requires deeper concurrency analysis. SCALEVIEW is unable to detect such instances whose root cause relates to the multi-threading strategy.

On top of the DCFs detected by SCALEVIEW, SCALEPICK successfully identifies anti-patterns for 36 out of 38 cases, demonstrating high effectiveness. In the remaining 2 cases, the DCFs are enclosed in inner classes implementing the `java.lang.Runnable` interface. Such classes are usually designed for multi-threading, being submitted to thread pools for executors, and are not handled by the call graph analysis in SCALEPICK. We leave as future work the detection of scalability faults related to multi-threading.

We also compare SCALELENS and SFIND in their capability to detect DCFs (SFIND does not detect anti-patterns). As shown in Table II, SFIND identified DCFs for only 9 out of the 55 faults, all of which involve explicit iterations. Compared to SCALELENS, SFIND detected 29 fewer DCFs. Even restricting our comparison to explicit iterations, SCALELENS still outperforms SFIND by detecting 18 more DCFs.

There are two reasons why SCALEVIEW, the dynamic analysis component of SCALELENS, outperforms SFIND in DCF detection. First, SFIND is designed to monitor the state of the system by performing heap measurement, which makes it unable to detect scale-dependent data structures that are not persistent when dimensions scale. In contrast, SCALEVIEW directly instruments the source code to obtain DCFs. Second, by design, SFIND only considers explicit application iterations, whereas SCALEVIEW takes into account both explicit application or library iterations and implicit iterations.

In addition to detecting more faults, SCALELENS provides richer diagnostic information. While SFIND only reports the

TABLE III: Ablation Analysis of SCALELENS.

|                             | Cassandra   | HDFS        | Ignite      |
|-----------------------------|-------------|-------------|-------------|
| # Fragments                 | 68,966      | 74,737      | 190,059     |
| # Fragments + Anti-Patterns | 7,823       | 16,982      | 12,467      |
| # DCFs                      | 251 (0.36%) | 106 (0.14%) | 342 (0.18%) |
| # DCFs + Anti-Patterns      | 129 (0.19%) | 68 (0.09%)  | 137 (0.07%) |

presence of a scale-dependent loop, SCALELENS reveals (1) the system dimension associated with the DCF, and (2) the anti-pattern underlying the DCF. These insights are crucial for developers to understand the root cause of scalability faults.

**Response to RQ1:** SCALELENS detects 29 more (4.2×) DCFs than the baseline, and fully identifies the DCFs and anti-patterns for 36 out of 55 previously known scalability faults. SCALELENS also reveals richer diagnostic information, such as the correlated system dimensions.

### C. RQ2: Finding Unknown Scalability Faults

In RQ1, we discussed the effectiveness of SCALELENS in detecting previously known scalability faults included in our empirical study. The goal of RQ2 is to evaluate the effectiveness of SCALELENS in detecting previously unknown scalability faults not present in our study. For that, we focus this evaluation on the latest stable versions of Cassandra (4.1.0), HDFS (3.4.0), and Ignite (2.16.0).

1) *Ablation Analysis & Manual Inspection*: SCALELENS combines SCALEVIEW and SCALEPICK. To assess the contribution of each component, we analyzed the results from running SCALELENS on the latest stable versions of Cassandra, HDFS, and Ignite, as detailed in Table III.

Without the analysis of SCALELENS, all loops and methods in the codebase are potential DCFs, leading to a large number of code fragments: 68,966 in Cassandra, 74,737 in HDFS, and 190,059 in Ignite.<sup>1</sup> Simply applying SCALEPICK to detect static code anti-patterns results in 7,823 to 16,982 code fragments, still making manual inspection impractical.

SCALEVIEW significantly reduces the number of code fragments of interest from 68,966 to 251 for Cassandra, representing 0.36% of the whole system. Out of the 251 DCFs pinpointed, SCALEPICK further identifies 129 that are associated with anti-patterns. Similarly, SCALEVIEW reduces the number of code fragments from 74,737 to 106 for HDFS, and from 190,059 to 342 for Ignite, with 68 and 137 DCFs containing anti-patterns, respectively. To confirm the correctness of the DCFs, we manually inspected all DCFs by plotting the iteration count to verify the increase of computational overhead along with the increase of the scale of the flagged dimension. For the DCFs with anti-patterns, we located the anti-patterns in the source code and confirmed their existence.

2) *Scalability Faults in the Wild*: SCALELENS identifies 129, 68, and 137 DCFs associated with anti-patterns in the latest versions of Cassandra, HDFS, and Ignite, respectively.

<sup>1</sup>Loops represent explicit iteration while a method called multiple times would capture implicit iteration.

From them, we found that 28 DCFs from Cassandra and 7 from HDFS are related to (de)serialization operations. Notably, all of them are (1) labeled as having linear growth to the dimension, and (2) identified with the *compute-cross* anti-pattern, which is correct as such operations are known to be IO-intensive due to their purposes. Such (de)serialization operations are often considered as necessary costs in distributed systems and are often unavoidable without major redesigns. As such, we exclude them from reporting. For the remaining DCFs, we manually confirmed that all of them exhibit problematic behavior and should be reported to developers. To contribute to the effort of reporting previously unknown scalability faults, we have reported 27 DCFs with anti-patterns to the developers of Cassandra and HDFS. So far, the developers have confirmed problematic behavior for 5 of them and are investigating 4 others. The rest are pending review and none have been marked as invalid. We are in the process of reporting the rest of the scalability faults.

**Response to RQ2:** By combining dynamic and static analyses, SCALENS enables efficient detection of scalability faults. SCALENS has detected previously unknown faults in the latest stable versions of Cassandra, HDFS, and Ignite, resulting so far in 27 reports to developers, with 5 confirmed and 4 under investigation.

## V. THREATS TO VALIDITY

### A. Construct Validity

The central constructs of our work are *scalability fault*, *scalable system dimension*, *DCF*, and *anti-pattern*. We defined each at first use and anchored our definitions in prior literature where possible: scalability fault follows Leesatapornwongsa et al. [58], and scalable system dimensions follow the scalability characterization framework of Duboc et al. [50, 51]. Alternative operational definitions are possible. To guard against inconsistency, we applied the resulting taxonomy uniformly across all 444 faults and released the full catalog of labeled issues for independent audit.

We use iteration count as the observable proxy for scale-dependent execution cost. However, some scale-sensitive costs do not manifest as loops, such as a recursive descent whose depth correlates with a dimension, or a chain of conditional branches whose path space grows with cluster size. To reduce the risk of missing such cases, SCALEVIEW tracks not only explicit iteration but also implicit iteration, where the number of method invocations across homogeneous components grows with the system's scale (Section III-A2).

Last, a construct-validity concern is the fidelity of the manually authored scaling workloads. A workload is intended to drive one or more dimensions to a large value while exercising representative APIs; a badly designed workload may fail to do so. We mitigate this by requiring workloads to report their current scale state through a *set-scale-state* API provided by SCALEVIEW (Section III-A1), which gives SCALEVIEW a runtime signal for whether the intended dimension is actually being scaled, and by releasing the full set of workloads used in our evaluation for independent inspection.

### B. Internal Validity

Our empirical findings rest on the manual selection and categorization of 444 scalability faults from over 199K issue reports. This manual process can introduce bias. We mitigate this with a multi-stage adjudication protocol: every issue is independently reviewed and tagged by two authors, a third author reviews every disagreement or uncertainty, and unresolved conflicts are settled in group discussion. We further mitigate the threat by publishing the full set of tagged issues and the tagging guidelines.

We also considered keyword-based selection as an alternative to this manual process. A broad keyword search over the 199K issues, using terms such as “scale”, “scalability”, “cluster”, “size”, “degradation”, “contention”, “out of memory”, “performance”, “regression”, and “impact”, returned 10,046 candidates, of which manual inspection revealed a 99% false-positive rate. Backward validation against the manually selected 444 faults showed that 78% of them contain none of those keywords. Automating selection by keyword would therefore have both over-reported and under-reported scalability faults, and the manual process was the only reliable option available.

A further internal-validity concern is the inherent non-determinism of dynamic analysis: the JIT compiler, garbage collector, and operating system scheduler introduce run-to-run variation in SCALEVIEW's recorded iteration counts. We mitigate this by filtering growth trends across multiple scale states rather than relying on single-point measurements (Section III-A4), so that the correlation between iteration count and scale state is robust to local noise.

### C. External Validity

The 10 systems in our empirical study (Section II) are all Java-based and span databases, file systems, compute frameworks, streaming platforms, and coordination services. This covers the most common categories of open-source distributed systems, but our findings may not transfer to non-JVM distributed systems (e.g., C++, Go, or Rust) or to application domains with substantially different scaling profiles, such as HPC-style tightly coupled compute.

SCALENS has two components with different generalization properties. SCALEVIEW instruments code at the JVM bytecode level and is therefore applicable to any Java-based system without source modification. SCALEPICK, in contrast, depends on static call-graph analysis, which is accurate for code with conventional control flow (explicit method calls, inheritance, explicit locks) but is less precise on event-driven, callback-heavy, reflective, or executor-dispatched code, where the call graph is incomplete or statically unresolvable. In our evaluation, for instance, two anti-pattern false negatives trace to Runnable-based executor code, which our analysis does not follow into the thread pool. In practical terms, SCALENS is most effective on Java batch, storage, and database systems such as Cassandra, HDFS, and Ignite, where control flow and synchronization relationships are largely recoverable from the source. Extending SCALENS to event-driven architectures

would likely require hybrid static-dynamic call-graph recovery, which we leave as future work.

SCALELENS's detection performance additionally depends on the coverage of the supplied scaling workloads. A DCF is detected only if the relevant dimension is exercised, so workloads that omit a dimension will not surface DCFs that depend on it. This is a generalization threat rather than a tool bug. Users of SCALELENS can mitigate it by authoring workloads that jointly scale the dimensions they care about, and by reusing workloads across versions; in our evaluation the same workloads were reused across 7 versions of Cassandra, 5 of HDFS, and 3 of Ignite at no additional cost.

For the tool evaluation, our benchmark comprises 55 scalability faults drawn from Cassandra, HDFS, and Ignite, which together account for 217 (48.87%) of the faults in our study. This gives substantial coverage but is not exhaustive: the remaining seven studied systems and fault categories outside *compute* and *unbound* are not represented in the benchmark. We additionally explored using the 10-bug benchmark from SCALECHECK [77], but those systems (Cassandra 1.1.x and 1.2.x, HDFS 2.0.0) were released before April 2013 and are no longer buildable due to outdated dependencies; we therefore report numbers only against our own benchmark.

Finally, the taxonomy itself carries an external-validity concern. The 4 root-cause categories and the 11 anti-patterns emerged from 444 faults in 10 systems, so additional systems or application domains, particularly outside the Apache Java ecosystem, may surface different categories or anti-patterns.

## D. Conclusion Validity

SCALEVIEW finds DCFs as system facts: a DCF is by construction an iterative code fragment whose iteration count correlates with a scaled dimension, so SCALEVIEW produces no false positives for DCF identification. SCALEVIEW may, however, miss DCFs whose dimension is not exercised by the supplied workload, a source of false negatives already discussed above.

SCALEPICK flags a DCF with an anti-pattern only when its slice contains a checker-specific sink (Section III-B). We manually verified every flagged anti-pattern on our benchmark and observed no false positives. The two anti-pattern false negatives on known faults both trace to Runnable-based executor code, tying back to the static-analysis limitation noted under External Validity.

Our headline effectiveness numbers, 36 fully detected and 2 partially detected out of 55 known scalability faults and 4.2× more DCFs than SFIND, are reported against this benchmark. They support our claims about SCALELENS's detection capability on the studied fault categories and systems, but should not be read as universal guarantees across arbitrary distributed systems. The 334 DCFs with anti-patterns reported on the latest stable versions of Cassandra, HDFS, and Ignite were individually verified by manual inspection, so the positive count is reliable, but the proportion that developers will ultimately confirm as problematic is still evolving as reports progress through upstream review.

## VI. RELATED WORK

SCALECHECK [77] discovers scalability bugs in large-scale distributed systems via two components: SFIND, a program analysis tool that identifies scale-dependent loops (those whose number of iterations grow alongside system-scale data structures), and STEST, a set of colocation techniques that emulate a real-scale cluster on a single machine. SCALELENS differs from SCALECHECK in three key ways. First, SCALEVIEW generalizes scale-dependent loops into DCFs by capturing implicit iteration, even when no explicit loop exists in the source. Second, while SFIND tracks heap-allocated collections to find loops, SCALEVIEW instruments bytecode directly; this allows it to detect DCFs involving transient data structures that a heap trace would miss. Finally, whereas SCALECHECK relies on STEST to exercise the identified loops through cluster emulation, SCALELENS provides SCALEPICK, a set of static checkers to identify anti-patterns known to incur in scalability issues when combined with DCFs. These approaches are complementary: SCALEPICK offers immediate root-cause analysis, while STEST reproduces problematic symptoms at runtime.

Other tools also follow this symptom-oriented emulation approach [52, 63, 80, 86]. For example, EXALT [80] aims to emulate hundreds of HDFS nodes in a single machine to observe I/O-related increases in processing time, while DIECAST [52] aims to emulate large networks using a single machine and observe how networking speed affects processing time. In contrast, SCALELENS does not require explicit symptoms nor emulation techniques as it detects unknown scalability faults based on a combination of DCFs and anti-patterns.

Other works utilize extrapolation [54, 57, 72, 90, 92] to project system behavior (e.g., execution time) at scale using smaller-scale measurements. PATTERNMINER [72] identifies scalability bottlenecks in centralized distributed systems (e.g., HDFS) by detecting repeating behavior and noting that such behavior repeats at larger scales. VRISHA [90], SCALANA [54], AUTOMATED [57] and [92] model the scalability of HPC applications combining smaller-scale behavior observations with techniques such as canonical correlation analysis, performance graphs, stack trace analysis and machine learning, respectively. SCALELENS differs in that it requires no specification of symptoms/behaviors, is architecture-agnostic, and focuses on distributed systems.

Finally, other works present techniques for detecting memory bloat [46, 53, 59, 64, 65], resource leaks [56, 70, 79, 81, 82], thread contention [41, 42, 61, 62, 85, 87, 91], chatty I/O [29, 44, 69, 84], and performance anti-patterns [47, 48, 60, 66, 67, 68, 71, 73, 74, 75, 78, 89]. We consider all these works orthogonal to SCALELENS, as they are limited to design flaws regardless of system scale. SCALELENS, however, focuses on scalability faults unique to large-scale distributed systems, identifying anti-patterns whose negative impact remains latent at small scales but manifests at large scales due to growth in one or more system dimensions.

## VII. CONCLUSION

To better understand scalability faults, we conducted a systematic study of 444 scalability faults from 10 distributed

systems. Our study uncovered the notion of DCFs, and various anti-patterns that result in scalability faults when combined with DCFs. Based on this, we developed SCALENS, a novel approach that combines dynamic and static analyses to detect scalability faults. Our evaluation showed that SCALENS effectively finds previously known scalability faults (36 out of 55), outperforms the baseline (4.2× more DCFs), and correctly detects 334 DCFs associated with anti-patterns in the latest stable versions of Cassandra, HDFS, and Ignite. The source code of SCALENS is publicly available at <https://github.com/ucd-plse/scalelens>. The empirical study data, evaluation data, and reproduction instructions are at <https://github.com/ucd-plse/scalability>.

#### ACKNOWLEDGMENT

This work is supported in part by NSF grants CCF-2119348 and CCF-2119184. Results presented in this paper were obtained using the Chameleon testbed [55] supported by the National Science Foundation.

#### REFERENCES

- [1] CA-15013. <http://issues.apache.org/jira/browse/CASSANDRA-15013>.
- [2] CA-15400. <http://issues.apache.org/jira/browse/CASSANDRA-15400>.
- [3] CA-5456. <http://issues.apache.org/jira/browse/CASSANDRA-5456>.
- [4] CA-5506. <http://issues.apache.org/jira/browse/CASSANDRA-5506>.
- [5] HA-15696. <http://issues.apache.org/jira/browse/HADOOP-15696>.
- [6] HA-16385. <http://issues.apache.org/jira/browse/HADOOP-16385>.
- [7] HD-13039. <http://issues.apache.org/jira/browse/HDFS-13039>.
- [8] HD-13768. <http://issues.apache.org/jira/browse/HDFS-13768>.
- [9] IG-8681. <http://issues.apache.org/jira/browse/IGNITE-8681>.
- [10] KF-5642. <http://issues.apache.org/jira/browse/KAFKA-5642>.
- [11] ST-3256. <http://issues.apache.org/jira/browse/STORM-3256>.
- [12] YR-6188. <http://issues.apache.org/jira/browse/YARN-6188>.
- [13] YR-7147. <http://issues.apache.org/jira/browse/YARN-7147>.
- [14] Summary of the Amazon Kinesis Event in the Northern Virginia (US-EAST-1) Region. <https://aws.amazon.com/message/11201/>.
- [15] Summary of the AWS Service Event in the Northern Virginia (US-EAST-1) Region. <https://aws.amazon.com/message/12721/>.
- [16] Summary of the Amazon DynamoDB Service Disruption and Related Impacts in the US-East Region. <https://aws.amazon.com/message/5467D2/>.
- [17] Apache Cassandra. <https://cassandra.apache.org/>. Accessed: 2026-04.
- [18] Apache Hadoop. <https://hadoop.apache.org/>. Accessed: 2026-04.
- [19] Apache HBase. <https://hbase.apache.org/>. Accessed: 2026-04.
- [20] Apache HDFS. [https://hadoop.apache.org/docs/r1.2.1/hdfs\\_design.html](https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html). Accessed: 2026-04.
- [21] Apache Ignite. <https://ignite.apache.org/>. Accessed: 2026-04.
- [22] Apache Kafka. <https://kafka.apache.org/>. Accessed: 2026-04.
- [23] Apache Hadoop MapReduce. <https://hadoop.apache.org/docs/current/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>. Accessed: 2026-04.
- [24] Apache Spark. <https://spark.apache.org/>. Accessed: 2026-04.
- [25] Apache Storm. <https://storm.apache.org/>. Accessed: 2026-04.
- [26] Apache Hadoop YARN. <https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/YARN.html>. Accessed: 2026-04.
- [27] Apache Projects Directory. <https://projects.apache.org/projects.html?category>.
- [28] Cassandra at Apple: 1000s of Clusters, 300k Nodes, 100 PB. <https://news.ycombinator.com/item?id=33124631>.
- [29] Performance Antipattern: Chatty I/O. <https://docs.microsoft.com/en-us/azure/architecture/antipatterns/chatty-io/>.
- [30] DB performance issue. <https://circleci.statuspage.io/incidents/hr0mm9xmmm3x6>.
- [31] CodeQL. <https://codeql.github.com/>.
- [32] Performance Antipattern: Extraneous Fetching antipattern. <https://docs.microsoft.com/en-us/azure/architecture/antipatterns/extraneous-fetching/>.
- [33] Cloud Filestore ListInstances API failed with error code 429 globally. <https://status.cloud.google.com/incidents/X8SNkK2BPYCrC1sveeu>.
- [34] BigQuery is experiencing issues with streaming API in US region. <https://status.cloud.google.com/incidents/mREMLwZF3FuLLn3zfTw>.
- [35] Javassist Github Repository. <https://github.com/jboss-javassist/javassist>.
- [36] Java Collections. <https://docs.oracle.com/javase/7/docs/api/java/util/Collections.html>.
- [37] Java Instrumentation Agents. <https://docs.oracle.com/javase/8/docs/technotes/guides/instrumentation/index.html>.
- [38] How LinkedIn customizes Apache Kafka for 7 trillion messages per day. <https://www.linkedin.com/blog/engineering/open-source/apache-kafka-trillion-messages>.
- [39] Kerberizing Hadoop Clusters at Twitter. [https://blog.x.com/engineering/en\\_us/topics/infrastructure/2023/kerberizing-hadoop-clusters-at-twitter](https://blog.x.com/engineering/en_us/topics/infrastructure/2023/kerberizing-hadoop-clusters-at-twitter).
- [40] The YARN Timeline Service v.2. <https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/TimelineServiceV2.html>.
- [41] M. Ahn, J. Han, Y. Kwon, and J. Jeong. Identifying on-/off-cpu bottlenecks together with blocked samples. In *OSDI*, pages 893–910. USENIX Association, 2024.
- [42] M. M. U. Alam, T. Liu, G. Zeng, and A. Muzahid. Syncperf: Categorizing, detecting, and diagnosing synchronization performance bugs. In *EuroSys*, pages 298–313. ACM, 2017.
- [43] B. Aronov, S. Har-Peled, C. Knauer, Y. Wang, and C. Wenk. Fréchet distance for curves, revisited. In *ESA*, volume 4168 of *Lecture Notes in Computer Science*, pages 52–63. Springer, 2006.
- [44] A. Avritzer, A. Janes, C. Trubiani, H. Rodrigues, Y. Cai, D. S. Menasché, and Á. J. A. de Oliveira. Architecture and performance anti-patterns correlation in microservice architectures. In *ICSA*, pages 60–71. IEEE, 2025.
- [45] D. W. Binkley and K. B. Gallagher. Program slicing. *Adv. Comput.*, 43:1–50, 1996.
- [46] Y. Bu, V. R. Borkar, G. Xu, and M. J. Carey. A bloat-aware design for big data applications. In *ISMM*, pages 119–130. ACM, 2013.
- [47] B. Chen, Z. M. Jiang, P. Matos, and M. Lacaria. An industrial experience report on performance-aware refactoring on a database-centric web application. In *ASE*, pages 653–664. IEEE, 2019.
- [48] T. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. N. Nasser, and P. Flora. Detecting performance anti-patterns for applications developed using object-relational mapping. In *ICSE*, pages 1001–1012. ACM, 2014.
- [49] N. Dayan and S. Idreos. Dostoevsky: Better space-time trade-offs for lsm-tree based key-value stores via adaptive removal of superfluous merging. In *SIGMOD Conference*, pages 505–520. ACM, 2018.
- [50] L. Duboc, D. S. Rosenblum, and T. Wicks. A framework for modelling and analysis of software systems scalability. In *ICSE*, pages 949–952. ACM, 2006.
- [51] L. Duboc, D. S. Rosenblum, and T. Wicks. A framework for characterization and analysis of software system scalability. In *ESEC/SIGSOFT FSE*, pages 375–384. ACM, 2007.
- [52] D. Gupta, K. V. Vishwanath, and A. Vahdat. Diecast: Testing distributed systems with an accurate scale model. In *NSDI*, pages 407–422. USENIX Association, 2008.
- [53] K. Jezek and R. Lipka. Antipatterns causing memory bloat: A case study. In *SAVER*, pages 306–315. IEEE Computer Society, 2017.

- [54] Y. Jin, H. Wang, X. Tang, Z. Guo, Y. Zhao, T. Hoefler, T. Liu, X. Liu, and J. Zhai. Leveraging graph analysis to pinpoint root causes of scalability issues for parallel applications. *IEEE Trans. Parallel Distributed Syst.*, 36(2):308–325, 2025.
- [55] K. Keahey, J. Anderson, Z. Zhen, P. Riteau, P. Ruth, D. Stanzione, M. Cevik, J. Colleran, H. S. Gunawi, C. Hammock, J. Mambretti, A. Barnes, F. Halbach, A. Rocha, and J. Stubbs. Lessons learned from the chameleon testbed. In *USENIX ATC*, pages 219–233. USENIX Association, 2020.
- [56] M. Kellogg, N. Shadab, M. Sridharan, and M. D. Ernst. Lightweight and modular resource leak verification. In *ES-EC/SIGSOFT FSE*, pages 181–192. ACM, 2021.
- [57] I. Laguna, D. H. Ahn, B. R. de Supinski, T. Gamblin, G. L. Lee, M. Schulz, S. Bagchi, M. Kulkarni, B. Zhou, Z. Chen, and F. Qin. Debugging high-performance computing applications at massive scales. *Commun. ACM*, 58(9):72–81, 2015.
- [58] T. Leesatapornwongsa, C. A. Stuardo, R. O. Suminto, H. Ke, J. F. Lukman, and H. S. Gunawi. Scalability bugs: When 100-node testing is not enough. In *HotOS*, pages 24–29. ACM, 2017.
- [59] B. Li, P. Su, M. Chabbi, S. Jiao, and X. Liu. Djxperf: Identifying memory inefficiencies via object-centric profiling for java. In *CGO*, pages 81–94. ACM, 2023.
- [60] J. Li, Y. Chen, H. Liu, S. Lu, Y. Zhang, H. S. Gunawi, X. Gu, X. Lu, and D. Li. Pcatch: automatically detecting performance cascading bugs in cloud systems. In *EuroSys*, pages 7:1–7:14. ACM, 2018.
- [61] N. Li, J. Guo, B. Huang, Y. Li, Y. Zhang, C. Li, and W. Huang. TCSA: efficient localization of busy-wait synchronization bugs for latency-critical applications. *IEEE Trans. Parallel Distributed Syst.*, 35(2):297–309, 2024.
- [62] R. Liscano, A. Ahmed, J. Robertson, A. Azim, V. Sundaresan, and Y. Chang. A lock contention classifier based on java lock contention anti-patterns. In *ICMLA*, pages 1106–1113. IEEE, 2023.
- [63] N. Machado, F. Maia, F. Neves, F. Coelho, and J. Pereira. Minha: Large-scale distributed systems testing made practical. In *OPODIS*, volume 153 of *LIPIcs*, pages 11:1–11:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019.
- [64] N. Mitchell, E. Schonberg, and G. Sevitsky. Four trends leading to java runtime bloat. *IEEE Softw.*, 27(1):56–63, 2010.
- [65] K. Nguyen, K. Wang, Y. Bu, L. Fang, and G. Xu. Understanding and combating memory bloat in managed data-intensive systems. *ACM Trans. Softw. Eng. Methodol.*, 26(4):12:1–12:41, 2018.
- [66] A. Nistor, L. Song, D. Marinov, and S. Lu. Toddler: detecting performance problems via similar memory-access patterns. In *ICSE*, pages 562–571. IEEE Computer Society, 2013.
- [67] A. Nistor, P. Chang, C. Radoi, and S. Lu. CAMEL: detecting and fixing performance problems that have non-intrusive fixes. In *ICSE (I)*, pages 902–912. IEEE Computer Society, 2015.
- [68] D. Petriu and G. Somadder. A pattern language for improving the capacity of layered client/server systems with multi-threaded servers. *Proceedings of EuroPLoP’97*, 1997.
- [69] R. Pincirol, A. Aleti, and C. Trubiani. Performance modeling and analysis of design patterns for microservice systems. In *ICSA*, pages 35–46. IEEE, 2023.
- [70] A. Shahoor, A. Y. Khamit, J. Yi, and D. Kim. Leakpair: Proactive repairing of memory leaks in single page web applications. In *ASE*, pages 1175–1187. IEEE, 2023.
- [71] S. Shao, Z. Qiu, X. Yu, W. Yang, G. Jin, T. Xie, and X. Wu. Database-access performance antipatterns in database-backed web applications. In *ICSME*, pages 58–69. IEEE, 2020.
- [72] R. Shi, Y. Gan, and Y. Wang. Evaluating scalability bottlenecks by workload extrapolation. In *MASCOTS*, pages 333–347. IEEE Computer Society, 2018.
- [73] C. U. Smith and L. G. Williams. New software performance antipatterns: More ways to shoot yourself in the foot. In *Int. CMG Conference*, pages 667–674. Computer Measurement Group, 2002.
- [74] C. U. Smith and L. G. Williams. Software performance antipatterns. In *Workshop on Software and Performance*, pages 127–136. ACM, 2000.
- [75] L. Song and S. Lu. Performance diagnosis for inefficient loops. In *ICSE*, pages 370–380. IEEE / ACM, 2017.
- [76] C. Soto-Valero, T. Durieux, and B. Baudry. A longitudinal analysis of bloated java dependencies. In *ESEC/SIGSOFT FSE*, pages 1021–1031. ACM, 2021.
- [77] C. A. Stuardo, T. Leesatapornwongsa, R. O. Suminto, H. Ke, J. F. Lukman, W. Chuang, S. Lu, and H. S. Gunawi. Scalecheck: A single-machine approach for discovering scalability bugs in large distributed systems. In *FAST*, pages 359–373. USENIX Association, 2019.
- [78] C. Trubiani, R. Pincirol, A. Biaggi, and F. A. Fontana. Automated detection of software performance antipatterns in java-based applications. *IEEE Trans. Software Eng.*, 49(4):2873–2891, 2023.
- [79] C. Wang, J. Liu, X. Peng, Y. Liu, and Y. Lou. Boosting static resource leak detection via llm-based resource-oriented intention inference. *CoRR*, abs/2311.04448, 2023.
- [80] Y. Wang, M. Kapritsos, L. Schmidt, L. Alvisi, and M. Dahlin. Exalt: Empowering researchers to evaluate large-scale storage systems. In *NSDI*, pages 129–141. USENIX Association, 2014.
- [81] G. Xu and A. Rountev. Precise memory leak detection for java software using container profiling. *ACM Trans. Softw. Eng. Methodol.*, 22(3):17:1–17:28, 2013.
- [82] G. Xu, M. D. Bond, F. Qin, and A. Rountev. Leakchaser: helping programmers narrow down causes of memory leaks. In *PLDI*, pages 270–282. ACM, 2011.
- [83] J. Yang, P. Subramaniam, S. Lu, C. Yan, and A. Cheung. How not to structure your database-backed web applications: a study of performance bugs in the wild. In *ICSE*, pages 800–810. ACM, 2018.
- [84] J. Yang, C. Yan, C. Wan, S. Lu, and A. Cheung. View-centric performance optimization for database-backed web applications. In *ICSE*, pages 994–1004. IEEE / ACM, 2019.
- [85] T. Yu and M. Pradel. Syncprof: detecting, localizing, and optimizing synchronization bottlenecks. In *ISSA*, pages 389–400. ACM, 2016.
- [86] Y. Zeng, M. Chao, and R. Stoleru. Emuedge: A hybrid emulator for reproducible and realistic edge computing experiments. In *ICFC*, pages 153–164. IEEE, 2019.
- [87] C. Zhang, J. Li, D. Li, and X. Lu. Understanding and statically detecting synchronization performance bugs in distributed cloud systems. *IEEE Access*, 7:99123–99135, 2019.
- [88] J. Zhang, F. Wang, S. Qiu, Y. Wang, J. Ou, J. Huang, B. Li, P. Fang, and D. Feng. Scavenger: Better space-time trade-offs for key-value separated lsm-trees. In *ICDE*, pages 4072–4085. IEEE, 2024.
- [89] G. Zhao, S. Georgiou, Y. Zou, S. Hassan, D. Truong, and T. Corbin. Enhancing performance bug prediction using performance code metrics. In *MSR*, pages 50–62. ACM, 2024.
- [90] B. Zhou, M. Kulkarni, and S. Bagchi. Vrisha: using scaling properties of parallel programs for bug detection and localization. In *HPDC*, pages 85–96. ACM, 2011.
- [91] F. Zhou, Y. Gan, S. Ma, and Y. Wang. wperf: Generic off-cpu analysis to identify bottleneck waiting events. In *OSDI*, pages 527–543. USENIX Association, 2018.
- [92] W. Zhou, J. Zhang, J. Sun, and G. Sun. Using small-scale history data to predict large-scale performance of HPC application. In *IPDPS Workshops*, pages 787–795. IEEE, 2020.